Лекция № 01. Структура CASE-средства. Структура среды разработки. Основные возможности

Структура CASE-средства. Структура среды разработки. Основные возможности.

Сегодня мы подробнее остановимся на том, из каких элементов состоит любое CASE – средство. **CASE-технология** (Computer – Aided Software Engineering, т.е. автоматизированная разработка ПО с помощью компьютерных технологий).

В настоящее время под термином CASE-средства понимаются программные средства, поддерживающие процессы создания и сопровождения ИС, включая анализ и формулировку требований, проектирование прикладного ПО (приложений) и баз данных, генерацию кода, тестирование, документирование, обеспечение качества, конфигурационное управление и управление проектом, а также другие процессы.

> **CASE-средства (от Computer Aided Software/System Engineering)** - позволяют проектировать любые системы на компьютере, необходимый элемент системного и структурно-функционального анализа, они позволяют моделировать бизнес-процессы, базы данных, компоненты программного обеспечения, деятельность и структуру организаций. Применимы практически во всех сферах деятельности.

> **Результат использования CASE-средств:**
> - оптимизация систем;
> - снижение расходов;
> - повышение эффективности;
> - снижение вероятности ошибок.

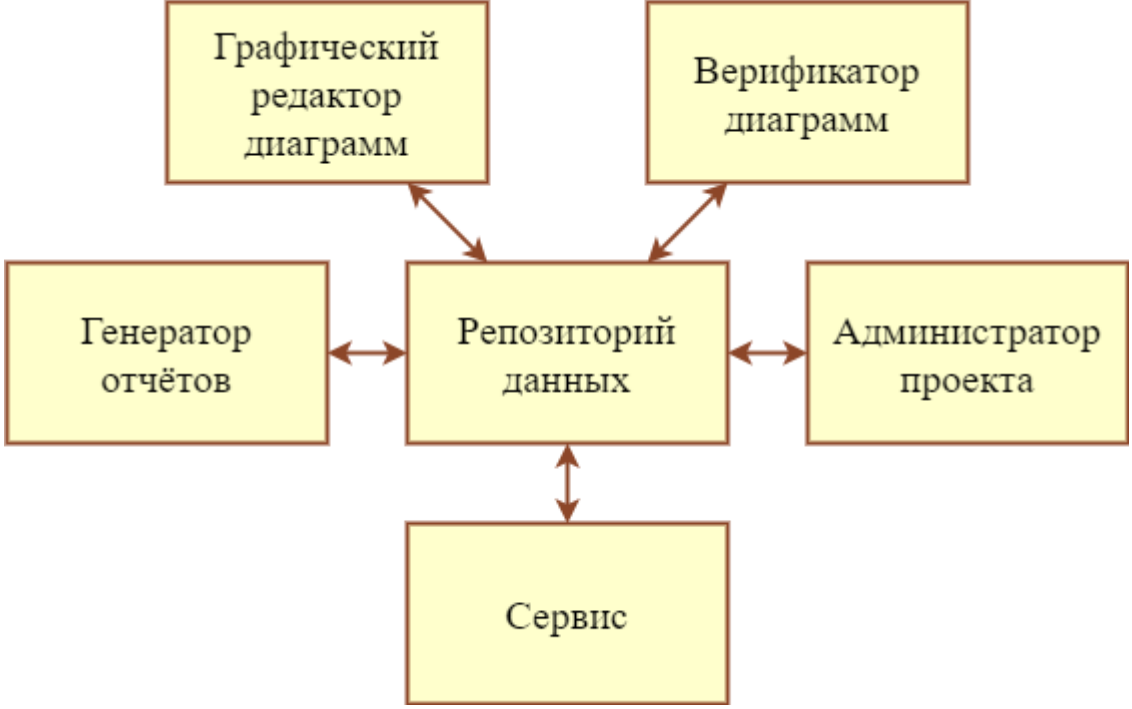
При разработке любой информационной системы в первую очередь возникает проблема взаимопонимания подрядчика и заказчика уже на стадии договоренности о структуре системы. Имея такой инструмент, как CASE-средство, проектировщик (аналитик) всегда может показать заказчику не абстрактное словесное описание процесса, а его конкретную модель (на экране ПК или в печатном виде).

В CASE-модели фаза прототипирования заменяет традиционную фазу **системного анализа**. Необходимо отметить, что наиболее автоматизируемыми фазами являются фазы контроля проекта и кодогенерации.

CASE-средства вместе с **системным ПО и техническими средствами** образуют **полную среду разработки ИС**.

Интегрированное CASE-средство (или комплекс средств, поддерживающих полный ЖЦ ПО) **содержит следующие компоненты:**

- **репозиторий**, являющийся основой CASE-средства. Он должен обеспечивать хранение версий проекта и его отдельных компонентов, синхронизацию поступления информации от различных разработчиков при групповой разработке, контроль метаданных на полноту и непротиворечивость;
- **графические средства анализа и проектирования**, обеспечивающие создание и редактирование иерархически связанных диаграмм (DFD, ERD и др.), образующих модели ИС;
- **средства разработки приложений**, включая языки 4GL и генераторы кодов;
- **средства конфигурационного управления;**
- **средства документирования;**
- **средства тестирования;**
- **средства управления проектом;**
- **средства реинжиниринга.**



Архитектуру CASE-средства

Основные возможности

В наиболее полном виде **CASE-средства обладают следующими характерными особенностями**:

1. **Единый графический язык**. CASE-технологии обеспечивают всех участников проекта, включая заказчиков, единым строгим, наглядным и интуитивно понятным графическим языком, позволяющим получать обозримые компоненты с простой и ясной структурой. При этом программы представляются двумерными схемами (которые проще в использовании, чем многостраничные описания), позволяющими заказчику участвовать в процессе разработки, а разработчикам — общаться с экспертами предметной области, разделять деятельность системных аналитиков, проектировщиков и программистов, облегчая им защиту проекта перед руководством, а также обеспечивая легкость сопровождения и внесения изменений в систему;
2. **Единая база данных проекта**. Основа CASE-технологии — использование базы данных проекта (репозитория) для хранения всей информации о проекте, которая может совместно использоваться разработчиками в соответствии с их правами доступа. Содержимое репозитория включает не только информационные объекты различных типов, но и отношения между их компонентами, а также правила использования или обработки этих компонентов. Репозиторий может хранить объекты различных типов: структурные диаграммы, определения экранов и меню, проекты отчетов, описания данных, логику обработки, модели данных, их организации и обработки, исходные коды, элементы данных и т. п.;
3. **Интеграция средств**. На основе репозитория осуществляются интеграция CASE-средств и разделение системной информации между разработчиками. При этом возможности репозитория обеспечивают несколько уровней интеграции: общий пользовательский интерфейс по всем средствам, передачу данных между средствами, интеграцию этапов разработки через единую систему представления фаз жизненного цикла, передачу данных и средств между различными платформами;
4. **Поддержка коллективной разработки и управления проектом**. CASE-технология поддерживает групповую работу над проектом, обеспечивая возможность работы в сети, экспорт-импорт любых фрагментов проекта для их развития и/ или модификации, а также планирование, контроль, руководство и взаимодействие, то есть функции, необходимые в процессе разработки и сопровождения проектов. Эти функции также реализуются на основе репозитория. В частности, через репозиторий могут осуществляться контроль безопасности (ограничения и привилегии доступа), контроль версий и изменений и т. п.;
5. **Макетирование**. CASE-технология дает возможность быстро строить макеты (прототипы) будущей системы, что позволяет заказчику на ранних этапах разработки оценить, насколько она устраивает его и приемлема для будущих пользователей;
6. **Генерация документации.** Вся документация по проекту генерируется автоматически на базе репозитория (как правило, в соответствии с требованиями действующих стандартов). Несомненное достоинство CASE-технологии заключается в том, что документация всегда отвечает текущему

состоянию дел, поскольку любые изменения в проекте автоматически отражаются в репозиторий (известно, что при традиционных подходах к разработке программного обеспечения документация в лучшем случае запаздывает, а ряд модификаций вообще не находит в ней отражения);
7. **Верификация проекта.** CASE-технология обеспечивает автоматическую верификацию и контроль проекта на полноту и состоятельность на ранних этапах разработки, что влияет на успех разработки в целом;
8. **Автоматическая генерация программного кода.** Генерация программного кода осуществляется на основе репозитория и позволяет автоматически построить до 85-90% текстов на языках высокого уровня.
9. **Сопровождение и реинжиниринг.** Сопровождение системы в рамках CASE-технологии характеризуется сопровождением проекта, а не программных кодов. Средства реинжиниринга и обратного инжиниринга позволяют создавать модель системы из ее кодов и интегрировать полученные модели в проект, автоматически обновлять документацию при изменении кодов, автоматически изменять спецификации при редактировании кодов и т. п.

Интегрированная среда разработки. Основные возможности и назначение.

Интегрированная среда разработки, ИСР (англ. IDE, Integrated Development Environment или Integrated Debugging Environment) — система программных средств, используемая программистами для разработки программного обеспечения (ПО).

Обычно среда разработки **включает в себя:**
- текстовый редактор;
- компилятор и / или интерпретатор;
- средства автоматизации сборки;
- отладчик.

Основным окном является текстовый редактор, который используется для ввода исходного кода в ИСР и ориентирован на работу с последовательностью символов в текстовых файлах. Такие редакторы обеспечивают расширенную функциональность – подсветку синтаксиса, сортировку строк, шаблоны, конвертацию кодировок, показ кодов символов и т. п. Иногда их называют редакторами кода, так как основное их предназначение – написание исходных кодов компьютерных программ.

Подсветка синтаксиса – выделение синтаксических конструкций текста с использованием различных цветов, шрифтов и начертаний. Обычно применяется в текстовых редакторах для облегчения чтения исходного текста, улучшения визуального восприятия. Часто применяется при публикации исходных кодов в Интернет.

Иногда содержит также средства для интеграции с системами управления версиями и разнообразные инструменты для упрощения конструирования графического интерфейса пользователя. Многие современные среды разработки также включают браузер классов, инспектор объектов и диаграмму иерархии классов — для использования при объектно-ориентированной разработке ПО.

Хотя и существуют ИСР, предназначенные для нескольких языков программирования — такие, как Eclipse, NetBeans, Embarcadero RAD Studio, Qt Creator или Microsoft Visual Studio, но обычно ИСР предназначается для одного определѐнного языка программирования - как, например, Visual Basic, PureBasic, Delphi, Dev-C++.

Частный случай ИСР, их эволюционное развитие — среды визуальной разработки, которые включают в себя возможность визуального редактирования интерфейса программы.

Основной единицей управления исходным кодом, используемой в различных IDE является **проект**. Как правило, также поддерживается некоторый способ объединения родственных проектов в группы.

Интегрированные среды разработки также часто поддерживают пометки в комментариях в исходном тексте программ, отмечающие места, требующие дальнейшего внимания или предполагающие внесение изменений, такие как TODO. В дальнейшем эти пометки могут выделяться редакторами (напр. vim, emacs,

встроенный редактор Visual Studio) или использоваться для организации совместной работы с построением тегов и задач (например, в IntelliJ). Использование комментариев с TODO так же является стандартом оформления кода на Object Pascal, Delphi.

> **Обычно интегрированная среда разработки** - это совокупность программных средств, поддерживающая все этапы разработки программного обеспечения от написания исходного текста программы до ее компиляции и отладки, и обеспечивающая простое и быстрое взаимодействие с другими инструментальными средствами (программным отладчиком-симулятором, внутрисхемным эмулятором, эмулятором ПЗУ и программатором).

Строго говоря, интегрированные среды разработки не относятся к числу средств отладки. Отладка – лишь одно из свойств интегрированных сред, которые представляют собой основу любой **визуальной среды разработки** или **RAD-среды**.

> **При традиционном подходе, начальный этап написания программы строится следующим образом:**
>
> 1. Исходный текст набирается при помощи какого-либо текстового редактора.
> 2. По завершении набора, работа с текстовым редактором прекращается и запускается кросс компилятор.
> 3. Как правило, вновь написанная программа содержит синтаксические ошибки, и компилятор сообщает о них на консоль оператора.
> 4. Вновь запускается текстовый редактор, и оператор должен найти и устранить выявленные ошибки, при этом сообщения о характере ошибок выведенные компилятором уже не видны, так как экран занят текстовым редактором.

И этот цикл может повторяться не один раз. Если программа имеет большой объем, собирается из различных частей, и подвергается длительному редактированию или модернизации, то даже этот начальный этап может потребовать много сил и времени. После этого наступает этап отладки программы и к редактору с компилятором добавляется эмулятор или симулятор, за работой которого хотелось бы следить прямо по тексту программы в текстовом редакторе.

Интегрированные среды (оболочки) разработки (Integrated Development Environment, IDE) позволяют **избежать большого объема однообразных действий** и тем самым существенно повысить эффективность процесса разработки и отладки позволяют, то есть они являются RAD-средами различной степени автоматизации процесса программирования.

> **Работа в интегрированной среде дает программисту**:
>
> - возможность использования встроенного многофайлового текстового редактора, специально ориентированного на работу с исходными текстами программ;
> - иметь автоматическую диагностику выявленных при компиляции ошибок, когда исходный текст программы, доступный редактированию, выводится одновременно с диагностикой в многооконном режиме;
> - возможность параллельной работы над несколькими проектами. Менеджер проектов позволяет использовать любой проект в качестве шаблона для вновь создаваемого проекта;
> - минимум перекомпиляции. Ей подвергаются только редактировавшиеся модули;
> - возможность загрузки отлаживаемой программы в имеющиеся средства отладки, и возможность работы с ними без выхода из оболочки;
> - возможность подключения к оболочке практически любых программных средств.

В последнее время, **функции интегрированных сред разработки** становятся **стандартной принадлежностью программных интерфейсов эмуляторов и отладчиков-симуляторов**.

Подобные функциональные возможности, в сочетании с дружественным интерфейсом, в состоянии существенно увеличить скорость разработки программ, особенно для микроконтроллеров и процессоров цифровой обработки сигналов, являющихся очень трудоемкими и труднообозримыми процессами.

Задание для самостоятельной работы:

1. Дать определение понятия транслятора, компилятора, отладчика, интерпретатора.
2. В чем заключается основной процесс отладки? (дать развернутый ответ).
3. Дайте определение системы управления версиями проекта. Приведите несколько основных терминов с определениями.
4. Опишите следующие ИСР: Eclipse, KDevelop, Microsoft Visual Studio по плану:
    - кто разработал, в каком году;
    - что лежит в основе ИСР;
    - на каком языке программирования написана;
    - основные функции;
    - отличия и особенности.
5. Приведите 5 примеров ИСР (кроме тех, которые упоминались в лекции) и кратко их опишите.